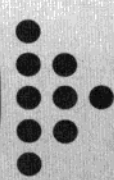




第16章

Java 注解

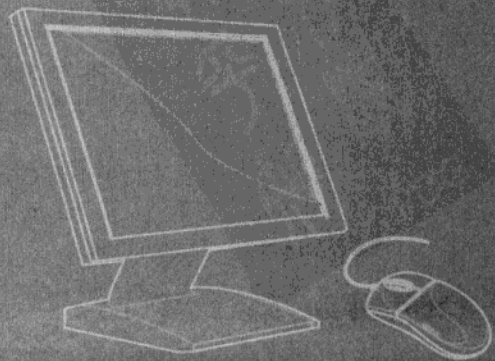


学前提示

JavaSE 5.0 以后的版本引入了一项新特性: Annotation, 中文翻译成注解, 是用来为程序元素(类、方法、成员变量等)设置说明和解释的一种元数据, Java 开发和部署工具可以读取这些注解, 并以某种形式处理这些注解。本文将从什么是注解、JavaSE 5.0 中内置的注解、如何自定义注解、如何对注解进行注解以及如何如何在程序中读取注解信息等五个方面进行讨论。

知识要点

- 注解是什么
- JavaSE 5.0 内置的基本注解类型
- 自定义注解类型
- 对注解进行注解
- 使用反射获取注解信息



16.1 注解概述

注解(annotation)是 JavaSE 5.0 以上版本新增加的功能。它可以添加到程序的任何元素上(包声明、类型声明、构造方法、方法、成员变量、参数),用来设置一些说明和解释,Java 开发和部署工具可以读取这些注释,并以某种形式处理这些注释,可生成其他 Java 编程语言源文件、XML 文档或要与包含注释的程序一起使用的其他构件。

在理解注解前,得先提一提什么是元数据(metadata)。元数据是用来描述数据的一种数据。从 JavaSE 5.0 开始,增加了元数据对 Java 源代码的描述,也就是注解。注解是代码里做的特殊标记,这些标记可以在编译、类加载、运行时被读取,并执行相应的处理。通过使用注解,程序员可以在不改变原有逻辑的情况下,在源文件中嵌入一些补充的描述源代码的信息(这些信息被存储在注解的“name=value”键值对中)。代码分析工具、开发工具和部署工具可以通过这些补充的描述源代码信息进行验证或者进行部署。注解类似于修饰符一样被使用,可以用于包、类、构造方法、方法、成员变量、参数、局部变量的声明。

需要注意的是,注解被用来为程序元素(类、方法、成员变量等)设置元数据,它不影响程序代码的执行,无论增加、删除注解程序的执行都不受任何影响。如果希望让程序中的注解起一定作用,只有通过配套的工具对注解中的元数据信息进行提取、访问,根据这些元数据增加额外功能和处理等。访问和处理注解的工具统称 APT(Annotation Processing Tool)。

16.2 JDK 内置的基本注解类型

Java 的注解采用“@”标记形式,后面跟上注解类型名称,如果注解需要数据,通过“name=value”向注解提供数据。例如, `@SuppressWarnings(value={"unchecked"})` 就是 SuppressWarnings 注解类型使用的一个示例。

注意

注解类型和注解的区别: 注解类型是某一类型注解的定义,类似于类。注解是某一注解类型的一个具体实例,类似于该类的实例。

在 JavaSE 5.0 的 java.lang 包中预定义了三个注解,分别是 Override、Deprecated 和 SuppressWarnings。下面分别讲解它们的含义。

16.2.1 重写 Override

Override 是一个限定重写方法的注解类型,用来指明被注解的方法必须是重写超类中的方法,这个注解只能用于方法上。编译器在编译源代码时会检查用 `@Override` 标注的方法是否有重写父类的方法。

先让我们来看看如果不用 Override 标识会发生什么事情。假设有两个类 Parent 和 Sub,在子类 Sub 中想重写父类 Parent 中的 myMethod 方法,但不小心把 Sub 类中的“myMethod”方法名写成了“mymethod”。



```

class Parent { //父类
    public void myMethod() {
        System.out.println("Parent.myMethod()");
    }
}
class Sub extends Parent { //子类继承父类
    public void mymethod() {
        System.out.println("Sub.myMethod()");
    }
}

```

如下，创建 Sub 的实例，并且调用 myMethod()方法。

```

/** JavaSE 5.0 内置注解类型: Override 的使用测试 */
public class OverrideTest {
    public static void main(String[] args) {
        Parent clazz = new Sub();
        clazz.myMethod();
    }
}

```

以上的代码可以正常编译通过和运行，但是输出的结果却不是我们想要的。因为在多态调用时，myMethod()方法并未被覆盖。当调用子类实例的 myMethod()方法时，实际调用到的还是父类 Parent 中的 myMethod()方法。更不幸的是，程序员并未意识到这一点。这就可能会产生 bug。

如果使用 Override 来修饰子类 Sub 中的 mymethod()方法，并描述此方法是要重写父类的 myMethod()方法的。此时，编译器就会报错。因此，就可以避免这类错误。

```

class Sub extends Parent { //子类继承父类
    @Override
    public void mymethod() {
        System.out.println("Sub.myMethod()");
    }
}

```

以上代码编译不能通过，被 Override 注解的方法必须在父类中存在同样的方法，程序才能编译通过。也就是说只有下面的代码才能正确编译。

```

class Sub extends Parent { //子类继承父类
    @Override
    public void myMethod() {
        System.out.println("Sub.myMethod()");
    }
}

```

16.2.2 警告 Deprecated

Deprecated 是用来标记已过时的成员的注解类型，用来指明被注解的方法是一个过时的方法，不建议使用了。当编译调用到被标注为 Deprecated 的方法的类时，编译器就会产生



在使用 Eclipse 等 IDE 编写 Java 程序时，经常会在属性或方法的提示中看到这个词。如果某个类成员的提示中出现了个词，就表示此处并不建议使用这个类成员。因为这个类成员在未来的 JDK 版本中可能被删除。之所以现在还保留，是为了给那些已经使用了这些类成员的程序一个缓冲期。如果现在就删除了，那么这些程序就无法在新的编译器中编译了。

说到这，您可能已经猜出来了，`Deprecated` 注解一定和这些类成员有关。说得对！使用 `Deprecated` 标注一个类成员后，这个类成员在显示上就会有一些变化。在 Eclipse 工具中就可以明显看到这个变化。让我们看看图 16.1 有哪些变化。

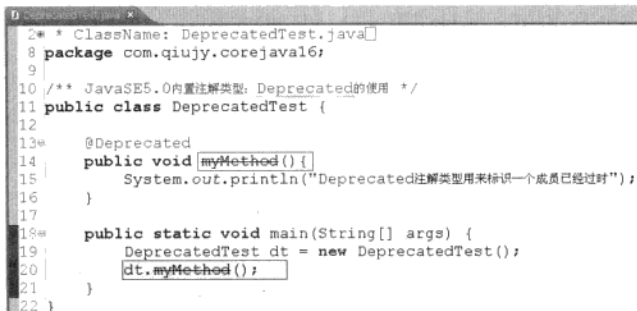


图 16.1 加上@Deprecated 后的类成员在 Eclipse 中的变化

从图中可以看出，红色框里面的就是变化了的部分。发生这些变化并不会影响编译，只是提醒一下程序员，这个方法以后是要被删除的，最好别用。

Deprecated 注解还有一个作用，就是如果一个类从另外一个类继承，并且重写了被继承类的 Deprecated 方法，在编译时将会出现一个警告。如 Class2.java 的内容如下。

```
class Class1{
    @Deprecated
    public void myMethod(){}
}

public class Class2 extends Class1{
    public void myMethod(){}
}
```

在命令行运行“javac Test.java”命令时，会出现如下警告。

注意: Class2.java 使用或覆盖了已过时的 API。

注意: 要了解详细信息, 请使用 `-Xlint:deprecation` 重新编译

使用-Xlint:deprecation 可显示更详细的警告信息，如下所示。

```
test.java:4: 警告: [deprecation] Class1 中的 myMethod() 已过时
    public void myMethod()
```

这些警告并不会影响编译，只是提醒一下尽量不要使用 `myMethod()` 方法。

16.2.3 抑制警告 SuppressWarnings

SuppressWarnings 是抑制编译器警告的注解类型，用来指明被注解的方法、变量或类在编译时如果有警告信息，就阻止警告。先看一看如下的代码片段。

```
public class SuppressWarningsTest {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("xxx");
    }
}
```

这是一个类中的方法。编译它，将会得到如下的警告。

注意：SuppressWarningsTest.java 使用了未经检查或不安全的操作。

注意：要了解详细信息，请使用 -Xlint:unchecked 重新编译。

这两行警告信息表示 List 类必须使用泛型才是安全的，才可以进行类型检查。如果想不显示这个警告信息有两种方法。一种是将这个方法进行如下改写。

```
public static void main(String[] args) {
    List<String> list = new ArrayList<String>(); // 通过泛型定义
    list.add("xxx");
}
```

另外一种做法就是使用 @SuppressWarnings 抑制警告信息。

```
/** JavaSE 5.0 内置注解类型: SuppressWarnings 的使用 */
public class SuppressWarningsTest {
    @SuppressWarnings("unchecked")
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add("xxx");
    }
}
```

注意

SuppressWarnings 和前两个注解不一样。这个注解有一个 value 属性，可以通过这个 value 属性来指定所有要抑制的警告类型名。

如 @SuppressWarnings(value={"unchecked", "deprecation"})，表示要抑制“未检查”警告和“已过时”警告。

16.3 自定义注解类型

注解的强大之处是它不仅可以使 Java 程序变成自描述的，而且允许程序员自定义注解类型。注解类型的定义和接口类型的定义差不多，只是在 interface 前面多了一个“@”。如：



```
/** 自定义注解类型 */
public @interface MyAnnotation {
}
```

上面的代码定义了一个最简单的注解类型，这个注解类型没有定义属性，也可以理解为一个标记注解。就像 `Serializable` 接口一样是一个标记接口，里面未定义任何方法。

当然，也可以定义带有属性的注解类型，如：

```
public @interface MyAnnotation{
    String value(); //定义一个属性
}
```

注解类型定义好之后，就可以按如下格式来使用了。

```
/** 使用自定义注解类型:MyAnnotation */
class UserMyAnnotation{
    @MyAnnotation("abc")
    public void myMethod(){
        System.out.println("使用自定义的注解");
    }
}
```

看了上面的代码，大家可能有一个疑问，怎么没有使用 `value`，而直接就写“abc”了。那么“abc”到底传给谁了？其实这里有一个约定，如果使用注解时没有显式指定属性名，却指定了属性值，而这个注解类型又有名为 `value` 的属性，就将这个值赋给 `value` 属性。如果没有在注解类型中定义名为 `value` 的属性，就会出现编译错误。

在定义注解类型时，还可以给它的属性指定默认值。

```
//定义自己的一个枚举类型
enum Status {ACTIVE, INACTIVE};
public @interface MyAnnotation{
    Status status() default Status.ACTIVE; //给 status 属性指定默认值
}
```

那么，在使用时，就可以不需要给 `status` 属性显式指定值了，它就会使用默认值。

```
/** 使用自定义注解类型:MyAnnotation */
class UserMyAnnotation{
    //value 属性的值为“abc”； status 属性使用默认值 Status.ACTIVE
    @MyAnnotation(value="abc")
    public void myMethod(){
        System.out.println("使用自定义的注解");
    }
}
```

当然，你还是可以给有默认的属性显式指定值的。

```
class UserMyAnnotation{
    @MyAnnotation(value="xxx", status=Status.INACTIVE)
    public void myMethod2(){
        System.out.println("使用自定义的注解");
    }
}
```



这里使用这个自定义注解类型时，给它的多个属性赋了值，多个属性之间用逗号“,”分隔。

这一节讨论了如何自定义注解类型。那么定义注解类型有什么用呢？有什么方法对注解类型的使用进行限制呢？能从程序中得到注解的信息吗？这些疑问都可以从下面的内容找到答案。

16.4 对注解进行注解

这一节的标题读起来虽然有些绕口，但它所蕴涵的知识却对设计更强大的 Java 程序有很大帮助。

在上一节讨论了自定义注解类型，由此可知，注解在 JavaSE 5.0 中也和类、接口一样，是程序的一个基本的组成部分。既然可以对类、接口进行注解，那么当然也可以对注解进行注解。JavaSE 5.0 中提供了四种专门用在注解上的注解类型，分别是 Target、Retention、Documented 和 Inherited。下面就分别介绍这四种特殊的注解类型。

16.4.1 目标 Target

Target 这个注解类型理解起来非常简单。target 的中文意思是“目标”，因此，您可能已经猜到这个注解类型和某一些目标相关。那么这些目标是指什么呢？

在了解如何使用 Target 之前，需要先认识另一个枚举 ElementType，这个枚举定义了注解类型可应用于 Java 程序的哪些元素。下面是 ElementType 的源代码。

```
package java.lang.annotation;
public enum ElementType{
    TYPE,                //适用于类、接口、枚举
    FIELD,               //适用于成员字段
    METHOD,               //适用于方法
    PARAMETER,           //适用于方法的参数
    CONSTRUCTOR,         //适用于构造方法
    LOCAL_VARIABLE,      //适用于局部变量
    ANNOTATION_TYPE,     //适用于注解类型
    PACKAGE              //适用于包
}
```

使用 Target 时，至少要提供这些枚举值中的一个，以指定这个注解类型可以应用于程序的哪些元素上。如：

```
package com.qiujiy.corejava16;

import java.lang.annotation.ElementType;
import java.lang.annotation.Target;

//表示自定义的这个注解类型只能作用在构造方法和成员方法上
@Target({ElementType.CONSTRUCTOR, ElementType.METHOD})
@interface MethodAnnotation{
```



```

}
/** 元注解 Target 的使用 */
@MethodAnnotation //作用在类上 -->编译出错
public class TargetTest {
    @MethodAnnotation //作用在方法上--> 正确
    public void myMethod(){
    }
}

```

以上代码定义了一个注解 `MyAnnotation` 和一个类 `TargetTest`，并且使用 `MyAnnotation` 分别对类 `Target` 和方法 `myMethod` 进行注解。如果编译这段代码是无法通过的。也许有些人感到惊讶，没错啊！但问题就出在 `Target` 上，由于 `Target` 使用了一个枚举类型属性，它的值是 `{ElementType.CONSTRUCTOR, ElementType.METHOD}`，这就表明 `MyAnnotation` 只能为方法注解，而不能为其他的程序元素进行注解。因此，`MyAnnotation` 自然也不能为类 `Target` 进行注解了。

说到这，大家可能已经基本明白了。原来 `Target` 所指的目标就是 Java 的程序元素，如类、接口、成员方法、构造方法等。

16.4.2 类型 Retention

既然可以自定义注解类型，当然也可以读取程序中的注解信息(如何读取注解信息将在下一节中讨论)。但是注解只有被保存在 class 文件中才可以被读出来。Java 编译器中处理类中出现的注解时，有以下 3 种方式。

- 编译器处理完后，不保留注解到编译后的类文件中。
- 将注解保留在编译后的类文件中，但是在运行时忽略它。
- 将注解保留在编译后的类文件中，并在第一次加载类时读取它。

这 3 种方式对应于 `java.lang.annotation.RetentionPolicy` 枚举的 3 个值，分别如下。

```

package java.lang.annotation;

public enum RetentionPolicy {
    SOURCE,        //编译器处理完后，并不将它保留到编译后的类文件中
    CLASS,         //编译器将注解保留在编译后的类文件中，但是在运行时忽略它
    RUNTIME        //编译器将注解保留在编译后的类文件中，并在第一次加载类时读取它
}

```

而 `Retention` 就是用来设置注解是否保存在 class 文件中的。下面的代码是 `Retention` 的详细用法。

```

@Retention(RetentionPolicy.SOURCE)
@interface MyAnnotation1 { }
@interface MyAnnotation2 { }
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnnotation3 { }

```

其中“`MyAnnotation1`”被注解为不保存在 class 文件中，它就像 Java 代码中的“`//`”注



释一样，在编译成字节码时会被过滤掉；“MyAnnotation2”没有使用 Retention，其他是使用了它的默认值“RetentionPolicy.CLASS”，它将被保存在 class 文件中，但在运行时会被忽略，也就是说在运行时使用反射是读取不到它的信息的；“MyAnnotation3”被注解为需要保存在 class 文件中，而且在运行时也可以通过反射来读取它的信息。

提示

JavaSE 5.0 内置的注解类型中的 Override、SuppressWarnings 的 RetentionPolicy 为 SOURCE，而 Deprecated 为 RUNTIME。

16.4.3 文档 Documented

顾名思义，Documented 这个注解类型和文档有关。默认的情况下，在使用 javadoc 自动生成文档时，注解将被忽略掉。如果想在文档中也包含注解，必须使用 Documented 定义。另外需要注意的是：定义为 Documented 的注解必须设置 Retention 的值为 RetentionPolicy.RUNTIME。如：

```
@Documented
@Retention(RetentionPolicy.RUNTIME)
@interface DocAnnotation {
}
```

16.4.4 继承 Inherited

继承是 Java 主要的特性之一。在类中的 protected 和 public 成员都将会被子类继承，但是父类上的注解会不会也被子类继承呢？很遗憾地告诉大家，在默认的情况下，父类上的注解并不会被子类继承。如果要让这个注解可以被子类上继承，就必须给这个注解类型定义上添加 Inherited 注解。如：

```
package com.qiujiy.corejava16;

import java.lang.annotation.Documented;
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface InheritedAnnotation {
    String name();
    String value();
}

@InheritedAnnotation(name="abc", value="bcd")
class Parent{
}
```



```
class SubClass extends Perent{
}
```

这里定义的 `InheritedAnnotation` 注解类型是 `Inherited` 的，当把这个注解作用于 `Perent` 类上时，它的子类 `SubClass` 也就相当于继承了这个注解。

注意，在实际应用中，很少把注解定义成这种行为，因为这样做会带来很多麻烦。

16.5 利用反射获取注解信息

前面讨论了如何自定义注解类型，但是自定义了注解类型又有什么用呢？也就是说，如何来获取这些注解中的信息，并用这些信息来完成一定功能？解决这个问题就需要使用前一章介绍的反射(reflect)机制。

前面介绍过，利用 Java 反射机制，可以在运行时动态地获取类的相关信息，如：类中的所有方法、所有属性、所有构造方法；还可以创建对象、调用方法等。同样利用反射也可以获取注解的相关信息。

首先要确认一点，反射是在运行时获取信息的。因此，要用反射获取注解的相关信息，这个注解必须是用 `@Retention(RetentionPolicy.RUNTIME)` 声明的。

JavaSE 5.0 API 中的 `java.lang.reflect.AnnotatedElement` 接口中定义了四种反射性读取注解信息的方法。

- `public Annotation getAnnotation(Class annotationType)`: 如果存在该元素的指定类型的注解，则返回这些注解，否则返回 `null`。
- `public Annotation[] getAnnotations()`: 返回此元素上存在的所有注解。
- `public Annotation[] getDeclaredAnnotations()`: 返回直接存在于此元素上的所有注解。
- `public boolean isAnnotationPresent(Class annotationType)`: 如果指定类型的注解存在于此元素上，则返回 `true`，否则返回 `false`。

`java.lang.Class` 类和 `java.lang.reflect` 包中的 `Constructor`、`Field`、`Method`、`Package` 类都实现了 `AnnotationElement` 接口，可以从这些类的实例上分别取得标注于其上的注解及相关信息。

【示例】下面通过一个示例演示如何使用反射读取自定义注解在使用时的信息。

首先，定义一个名为“`MyAnno`”的注解类型。因为这个注解需要运行时反射获取信息，所以指定为 `RetentionPolicy.RUNTIME` 的。

```
package com.qiujy.corejava16;
import java.lang.annotation.*;

//这个注解可以用于类、接口、枚举、方法之上且可以在运行时用反射来获取它的信息
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno{
    String value() default "无值"; //给 value 属性指定默认值
}
```



接下来,再定义一个 UserMyAnno 类来使用这个注解。这里把 MyAnno 注解应用在类上和方法中。

```
package com.qiujuy.corejava16;

@MyAnno
class UserMyAnno{ //在 UserMyAnno 类上使用 MyAnno 注解
    @MyAnno("method")
    @Deprecated
    public void test(){ //在 test 方法上使用 MyAnno 注解和 Deprecated 注解
        System.out.println("test");
    }
}
```

最后,通过反射来获取这两处使用 MyAnno 注解的信息。

```
package com.qiujuy.corejava16;

import java.lang.annotation.*;
import java.lang.reflect.Method;

/** 利用反射动态获取注解的信息 */
public class ReflectAnnotationInfo {
    public static void main(String[] args) throws SecurityException,
        NoSuchMethodException {
        //获取类上的指定注解的 Annotation 实例
        Annotation anno1 = UserMyAnno.class.getAnnotation(MyAnno.class);
        if(anno1 != null){
            MyAnno myAnno = (MyAnno)anno1;
            System.out.println("类上的 MyAnno 注解:value=" + myAnno.value());
        }
        //取得 test() 方法的对应的 Method 实例
        Method method = UserMyAnno.class.getMethod("test");
        //取得 test() 方法上所有的 Annotation
        Annotation[] annotations = method.getAnnotations();
        for(Annotation anno : annotations){
            System.out.println("注解类型名:" + anno.annotationType().getName());
        }
    }
}
```

运行这个程序,在控制台的输出结果为:

```
类上的 MyAnno 注解:value=无值
注解类型名:com.qiujuy.corejava16.MyAnno
注解类型名:java.lang.Deprecated
```

结果说明,使用反射已经正确获取了注解的相关信息。



16.6 本章练习

- (1) 编写一个 `Person` 类，使用 `Override` 注解它的 `toString` 方法。
- (2) 自定义一个名为 “`MyTiger`” 的注解类型，它只可以使用在方法上，带一个 `String` 类型的 `value` 属性，然后在某个第 1 题中的 `Person` 类上正确使用。再编写一个测试类反射读取这个注解的属性值。